

Vorlesung Rechnernetze

Theorieübung Socketbefehle und Pakete

Prof. Dr. Dirk Staehle

Die Abgabe erfolgt durch Hochladen der Lösung in Moodle und exemplarisches Vorrechnen in der Laborübung.

Bearbeitung in Zweier-Teams

Team-Mitglied 1:

Team-Mitglied 2:

In dieser Aufgabe geht es darum, den Zusammenhang von Socket-Befehlen und übertragenen Pakete zu verstehen. Dazu ist in Abbildung 1 ein Python-Code dargestellt. Dieser Python Code wird auf einem Rechner zweimal mit unterschiedlichen Konfigurationen nacheinander ausgeführt. Bei der ersten Ausführung (A) ist My_IP=127.0.0.2 und Remote_IP=127.0.0.1. Bei der zweiten Ausführung (B) ist My_IP=127.0.0.1 und Remote_IP=127.0.0.2.

Ein Mitschnitt der ausgetauschten Pakete ist in Tabelle 1 dargestellt.

1. Tragen Sie in die erste Spalte von Tabelle 1 ein, welche Code-Zeile bei welcher Ausführung des Codes (A oder B) diese Paketübertragung bewirkt hat, sofern diese explizite Zuordnung möglich ist d.h. wenn die Übertragung des Pakets unmittelbar durch eine Code-Zeile ausgelöst wird.
2. Tragen Sie in die zweite Spalte von Tabelle 1 ein, welche „blockierende“ Befehlszeile (connect, accept, recv) bei welcher Ausführung des Codes (A oder B) „beendet“ wird, wenn das Paket in dieser Zeile empfangen wird. Auch hier soll natürlich nur dann ein Eintrag erfolgen, wenn der Empfang dieses Pakets die „Fertigstellung“ eines Socket-Befehls bewirkt.

Hinweis: Die Eintragungen in die Tabelle sollen also beispielweise die Form A10 haben, wenn Programmzeile 10 der ersten Ausführung (A) des Codes die Übertragung des Pakets bewirkt hat, oder B10 wenn dementsprechend die zweite Ausführung (B) des Codes die Übertragung des Pakets bewirkt hat.

Abbildung 1 Code Listing

```
1  import socket
2  socket.setdefaulttimeout(30)
3
4  My_IP = '127.0.0.1'
5  My_PORT = 50000
6  Remote_IP='127.0.0.2'
7  Remote_PORT=50000
8
9  def start_task(sock,message):
10     sock.send(message.encode('utf-8'))
11     msg=sock.recv(1024)
12     sock.close()
13
14  def start_server():
15     sock=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
16     sock.bind((My_IP, My_PORT))
17     sock.listen(1)
18     try:
19         conn, addr = sock.accept()
20         start_task(conn,"Thx for connecting!!!")
21     except socket.timeout:
22         pass
23
24  sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
25  try:
26     sock.connect((Remote_IP, Remote_PORT))
27     start_task(sock,"Thx for accepting!!!");
28  except socket.error:
29     start_server()
```

Tabelle 1: TCPdump

Übertragung ausgelöst von Programmzeile	Empfang bewirkt "Fertigstellung " von Programmzeile	No.	Time	Source	Destination	Protocol	Length	Src Port	Dst Port	Info
		1	1,340	127.0.0.2	127.0.0.1	TCP	52	56835	50000	56835 > 50000 [SYN] Seq=0
		2	1,340	127.0.0.1	127.0.0.2	TCP	40	50000	56835	50000 > 56835 [RST, ACK] Seq=1 Ack=1
		3	1,842	127.0.0.2	127.0.0.1	TCP	52	56835	50000	[TCP Spurious Retransmission] 56835 > 50000 [SYN] Seq=0
		4	1,842	127.0.0.1	127.0.0.2	TCP	40	50000	56835	50000 > 56835 [RST, ACK] Seq=1 Ack=1
		5	2,342	127.0.0.2	127.0.0.1	TCP	48	56835	50000	[TCP Spurious Retransmission] 56835 > 50000 [SYN] Seq=0
		6	2,342	127.0.0.1	127.0.0.2	TCP	40	50000	56835	50000 > 56835 [RST, ACK] Seq=1 Ack=1
		7	3,646	127.0.0.1	127.0.0.2	TCP	52	56837	50000	56837 > 50000 [SYN, ACK] Seq=0 Ack=1
		8	3,646	127.0.0.2	127.0.0.1	TCP	52	50000	56837	50000 > 56837 [SYN, ACK] Seq=0 Ack=1
		9	3,646	127.0.0.1	127.0.0.2	TCP	40	56837	50000	56837 > 50000 [ACK] Seq=1 Ack=1
		10	3,646	127.0.0.1	127.0.0.2	TCP	60	56837	50000	56837 > 50000 [PSH, ACK] Seq=1 Ack=1 Len=20
		11	3,646	127.0.0.2	127.0.0.1	TCP	40	50000	56837	50000 > 56837 [ACK] Seq=1 Ack=21 Len=0
		12	3,646	127.0.0.2	127.0.0.1	TCP	61	50000	56837	50000 > 56837 [PSH, ACK] Seq=1 Ack=21 Len=21
		13	3,646	127.0.0.1	127.0.0.2	TCP	40	56837	50000	56837 > 50000 [ACK] Seq=21 Ack=22 Len=0
		14	3,646	127.0.0.1	127.0.0.2	TCP	40	56837	50000	56837 > 50000 [FIN, ACK] Seq=21 Ack=22 Len=0
		15	3,646	127.0.0.2	127.0.0.1	TCP	40	50000	56837	50000 > 56837 [FIN, ACK] Seq=22 Ack=22 Len=0
		16	3,646	127.0.0.1	127.0.0.2	TCP	40	56837	50000	56837 > 50000 [ACK] Seq=22 Ack=23 Len=0

